Unit 4: Implementation and Refinement

This unit focuses on the transition from a theoretical design (flowcharts and pseudocode) to a functional computer program. We will examine the mechanics of how instructions are structured and how we ensure they are correct.

4.1 Translating Design to Code

Implementation is the process of writing the algorithm in a specific programming language. While the syntax (grammar) varies between languages like **C** and **Python**, the underlying logic remains consistent.

4.1.1 Assignment Statements

An assignment statement gives a value to a variable.

- **Concept:** `Variable = Expression`
- **Logic:** The computer evaluates the right side first and stores the result in the location named on the left.
- **Example:** `Area = Length * Width`

4.1.2 Input and Output (I/O) Statements

For a program to be useful, it must interact with the user.

- **Input:** Getting data from the user (e.g., `scanf` in C, `input()` in Python).
- **Output:** Displaying results to the screen (e.g., `printf` in C, `print()` in Python).

4.1.3 Conditional (Selection) Statements

These allow the program to make decisions and follow different paths.

- **Structure:** `IF (Condition) THEN (Action A) ELSE (Action B)`
- **Example:** Checking if a student passed based on a score of 40.

4.2 Dry Running and Manual Tracing

Before running code on a computer, a programmer should perform a **Dry Run**.

- **Definition:** A manual technique where you "act like the computer" and follow the logic of the algorithm on paper.
- **Trace Table:** A tool used to track the values of variables as they change at each step of the algorithm. This is the most effective way to catch logic errors before they become bugs.

4.3 Evaluation and Testing

Once the code is written, it must be evaluated against the original problem requirements.

- **Functional Testing:** Does the program produce the correct output for valid inputs?
- **Edge Case Testing:** How does the program handle unusual inputs (e.g., entering a letter when a number is expected, or entering zero in a division problem)?
- **Error Types:**
    1. **Syntax Errors:** "Grammar" mistakes in the code that prevent it from running.
    2. **Runtime Errors:** Errors that occur while the program is running (e.g., dividing by zero).
    3. **Logic Errors:** The program runs, but the output is wrong because the underlying plan was flawed.

4.4 Refinement and Optimization

Refinement is the "polishing" phase. After a program works correctly, we look for ways to make it:

- **More Efficient:** Can it run faster or use less RAM?
- **More Readable:** Is the code well-commented and organized so other programmers can understand it?
- **More Robust:** Can it handle unexpected user behavior without crashing?